

Handling Imbalanced Data using Ensemble Learning in Software Defect Prediction

Ruchika Malhotra

Computer Science and Engineering Department
Delhi Technological University
Delhi, India
Ruchikamalhotra2004@yahoo.com

Juhi Jain

Computer Science and Engineering Department
Delhi Technological University
Delhi, India
erjuhijain@gmail.com

Abstract—With the ever growing software industry, software defect prediction is one of the key ingredients in recipe of producing good quality software. Defects uncovered well in time helps in saving resources in terms of time, effort and money. However imbalanced nature of software data may hamper the resultant performance of models leading to incorrect interpretations of results. This problem has dragged attention of researchers and many solutions are proposed to overcome the effect of this problem. This paper aims to provide empirical comparison of software defect prediction models developed by using various boosting based ensemble methods on three open source JAVA projects. Four ensemble methods incorporate resampling techniques within them. Performances of models obtained are evaluated using stable metrics like Balance, G-Mean and AUC. Results show that use of resampling techniques before classifying using ensemble method has significantly improved model prediction as compared to classic boosting models. RUSBoost is the undisputed winner amongst all followed by MSMOTEBoost and SMOTEBoost.

Keywords—software defect prediction; data imbalance problem; resampling techniques; ensemble methods; boosting

I. INTRODUCTION

Software is the heart of this technological era. If there are frequent defects in software, it reduces reliability and credibility of software resulting into unsatisfied customers deteriorating the company's image. If defects are uncovered in early stages of software development, then one can allocate resources to required software component only. So, early detection of defects not only save time and resources but also enhances company's goodwill increasing its market share [1]. Software defect prediction using machine learning has therefore gained researchers' attention in recent years with aim of timely delivery of softwares with optimal utilization of resources. But one of the major concerns in machine learning is dealing with imbalanced nature of software data [2]. Considering binary problems, accuracy is considered a good performance metric for comparison of models but what if number of instances of one class is much greater than that of another class? Let us say, out of 100 instances 90 are defect free and 10 have defects. When model is trained on such imbalanced data, will it affect the validation results? Of course, the results will be biased towards majority class. This class imbalance problem is prevalent in many real world problems.

One possible solution to this problem is use of ensemble methods. Aim of this study is to provide empirical comparison of predictive capabilities of machine learning models that are developed using broadly two types of ensemble methods taking C4.5 as base classifier. Research questions designed to achieve the addressed objective are:

RQ1: Do resampling techniques enhance the performance of boosting based ensemble methods in case of imbalance data in software world?

RQ2: Which boosting based ensemble method outperforms for prediction of software defects?

Static code attributes of three open source JAVA softwares are exploited to predict software defects. Three classic boosting ensembles used in this study are AdaBoost, AdaBoost.NC and AdaBoost.M1. Performances are then measured in terms of stable metrics- Balance, G-Mean and Area under Curve (AUC) and compared with that of four boosting based ensembles that involve data balancing strategies (RUSBoost, SMOTEBoost, MSMOTEBoost, DATABoost). Estimates of performance evaluators are gained by performing ten-fold cross validation for each dataset. Static code attributes of three open source JAVA softwares are exploited to predict software defects. Statistical validation of models developed is executed using nonparametric test-Friedman Test.

Rest of paper is arranged in four more sections. Section II reviews the related studies of this work. Research methodology and experimental design is examined in section III. Section IV discusses and analyses the results. Last, but not the least, section V concludes the empirical study performed and reveals the future directions.

II. RELATED WORK

Class imbalance problem is encountered in large number of real world problems. Some of them are fraud detection [3], risk management [4] and fault diagnosis [5] [6]. Object oriented (OO) metrics are widely used to explore software quality attributes in existing literature [7]. This work will be focused on application of boosting based ensembles using these static code attributes. Many algorithms are designed to handle imbalanced data. Tosun et al. [8] presented ensemble method that outperformed naïve bayes using balance as performance metric for NASA datasets. Sun et al. [9] proposed new

technique for handling class imbalance by converting binary class problem into multiclass problem. Wang and Yao [10] also analyzed different solution to class imbalance problem including resampling and ensembles over the NASA datasets and proposed a new variant of AdaBoost.NC. A novel overbagging based framework has been proposed for NASA datasets by Mousavi et al. [11] in which ensemble is dynamically selected. Huda et al. [12] also proposed the ensemble method based on three oversampling methods and performed comparative analysis with some existing strategies based on accuracy, recall and AUC. Song et al. [13] conducted the detailed experimentation and have included open source software data but not explored the performance based on Balance and AUC. Closest study to our work is Rodreguiz et al. [14] but they performed empirical comparison on NASA datasets where as we have used open source software projects. Also, they did not explore G-Mean and Balance performance evaluators that are considered stable in imbalanced data world.

III. RESEARCH METHODOLOGY

In this section, we notify about the source and nature of our datasets and dependent/independent variables of study followed by the incorporated data preprocessing steps. Next, various ensemble methods used to generate prediction models, performance evaluators employed to compare those models and statistical tests exploited to validate the models are explained.

A. Dataset Collection

This study exploits three open source Apache projects that are coded in JAVA language. Datasets used are curated by Marian Jureckjo [15] and are available on promise repository. Three datasets selected are Synapse 1.0, Camel 1.4 and Ant 1.0. Apache Synapse 1.0 is service oriented architecture (SOA) framework that provides web services. Camel 1.4 is JAVA framework that assists users to implement various design patterns for enterprise integration where as Ant 1.7 is flexible JAVA library used to build JAVA and non JAVA applications. Table I summarizes the datasets in terms of class distribution in increasing order of percentage of defective classes. Total number of classes in software is denoted by #Classes. #DC corresponds to number of defective classes and #NDC stand for number of defective classes. %age of DC signifies the actual proportion of defective classes in complete software. As is evident from table I, %age of DC varies from 10.19% to 22.28%. This represents highly imbalanced nature of software data.

B. Independent and Dependent Variables

Static code attributes viz. OO metrics of the project describes the internal software quality attributes and are used as independent variables in this study. In above described datasets, there are 21 independent variables comprising of CK metric suite [16], five metrics from QMOOD metric suite [17], two metrics from Martin [18], one metric from Henderson-Seller [19], three metrics based on McCabe's complexity, LOC metric and three additional metrics. Metrics' definitions can be referred from http://gromit.iar.pwr.wroc.pl/p_inf/ckim/metric.html.

TABLE I. DATASET DETAILS

Dataset	#Classes	#DC	#NDC	%age of DC
Synapse1.0	157	16	141	10.19
Camel1.4	872	727	145	16.63
Ant1.7	745	166	580	22.28

Defect is the binary dependent variable identified for this experimentation. Defect variable can have two possible values-0 for nondefective class and 1 for defective class. Original data set includes number of defects which can have multiple values. We convert it into binary variable-defect as desired for the purpose of our study.

C. Data Preprocessing

Datasets were checked for any missing data. Then two step data preprocessing is carried out for the datasets. First, outliers are identified and removed. Second, important features are selected amongst 21 OO metrics. There was no missing data in related datasets.

- **Outlier Removal:** If value of any instance lies far away from range of values of other instances, then that instance is considered outlier. Outliers are located in this study with help of inter quartile range. We used Waikato Environment for Knowledge Analysis (WEKA) tool for this purpose [20]. We identified 53, 50 and 125 outliers in Synapse 1.0, Camel 1.4 and Ant 1.7 respectively. We removed these outliers so that model validation is not affected by extremities.
- **Feature Selection:** After the outliers are removed, we focused on selecting the important features of the datasets. Performance of classifier is hampered by the presence of irrelevant or redundant features in feature space [21] [22]. We used correlation feature selection (CFS) that is proposed by [23]. Reference [22] also favors the application for CFS for measuring the goodness of features in imbalanced data. Selected features for respective datasets are noted in table II.

D. Ensemble Methods

Though there are many solutions to class imbalance problem, this study is limited to boosting algorithms in ensemble methods. Many classifiers are trained using training data and concluding judgment is based on their combination. C4.5 [25] is taken as base classifier for generating ensemble models. Boosting algorithm aims to improve the predictive capability of learner by recognizing the weak spots, i.e., instances that are wrongly classified. It amplifies the weightage of incorrect classified instances and new learner utilizes this improved dataset. Boosting creates several models that harmonize each other and prediction is done based on their weighted voting. AdaBoost is adaptive boosting algorithm that works on sampling framework [27] and its variant AdaBoost.NC is also incorporated. Adaboost.NC is adaptive boosting with negative correlation learning extension.

AdaBoost.M1 is another extension of AdaBoost for binary class problem.

TABLE II. SELECTED FEATURES OF DATASETS USING CFS

Dataset	Features Selected
Synapse 1.0	LOC, MAX CC, AMC, RFC, WMC
Camel 1.4	WMC, CBO, RFC, Ca, Ce, NPM, IC, CBM
Ant 1.7	WMC, RFC, LOC, CAM, AMC

In AdaBoost.M1 (AdaBM1) [26] ensemble method, as training starts, equal weights are allocated to training data. Weight of the instance is increased or decreased in next iteration depending on whether that instance is correctly classified or not. AdaBoost.M2 (AdaBM2) [27] is a slight variant of previous one which removes the restriction of maximum weight assigned to the instance. RUSBoost (RUSB) [28] is hybrid approach that employs random under sampling of instances before applying boosting. Other preprocessing techniques used are SMOTE, MSMOTE and data resampling. These corresponds to SMOTEBoost (SMOTEB) [29], MSMOTEBoost (MSMOTEB) [30] and DataBoost (DataB) [31] ensemble methods. SMOTEBoost is boosting algorithm based on oversampling technique- Synthetic Minority Oversampling Technique (SMOTE). Synthetic instances of minority classes are created and injected at each boosting iteration. RUSBoost works same as SMOTEBoost. Only difference is in resampling technique used. At each iteration of boosting, instead of SMOTE, random undersampling (RUS) is used. In DataBoost ensemble method, data generation is combined with boosting algorithm to improve the predictive capability of majority as well as minority class. First, hard instances are identified and then synthetic samples are generated based on these hard instances. These synthetic instances are reweighted and then added to training set of data.

E. Performance Evaluators

Many researchers have advocated that accuracy as a performance evaluator do not reflect the true classification picture for imbalanced data [32] [33]. So we have used more stable metrics like G-Mean, Balance and AUC. We also calculated one of traditional metric- sensitivity as it assists in understanding imbalanced nature of data. These metrics are calculated using confusion matrix comprising of true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN).

- Sensitivity: It is percentage of correctly predicted defective classes and is calculated as
$$Sensitivity = \frac{TP}{TP+FN} \quad (1)$$
- G-Mean: is geometric mean of sensitivity and specificity and is calculated as
$$GMean = \sqrt{Sensitivity \times Specificity} \quad (2)$$
- Balance: In ideal case, precision should be 1 and false positive rate (FPR) should be 0. So, Balance uses this definition to make a reliable performance evaluator for imbalanced data. It is calculated as

$$Balance = 1 - \sqrt{\frac{(0 - FPR)^2 + (1 - Precision)^2}{2}} \text{ where}$$

$$FPR = \frac{FP}{FP + TN} \text{ and } Precision = \frac{TP}{TP + FN} \quad (3)$$

- AUC: Area under Receiver operating characteristic (ROC) curve defines relationship between sensitivity and specificity and is well accepted performance metric.

F. Validation and Statistical Tests

Ten fold cross validation is used to evaluate each developed model. In this, a particular dataset is divided into ten approximately equal parts. Nine parts are used for training the model and remaining part is used for testing the model. This procedure is repeated for all ten partitions [34]. Employing ten fold cross validation while training the model builds the confidence in the prediction results as it reduces chance of overfitting.

We also used Friedman test [33] to statistically validate results of models for sensitivity, G-Mean, Balance and AUC. Friedman test is non parametric technique and, therefore, there is no need of any presumption to be satisfied by the datasets. We designed null hypothesis as: there is no difference between the predictive capabilities of seven boosting ensemble methods.

IV. RESULTS AND ANALYSIS

We have used open source Knowledge Extraction based on Evolutionary Learning (KEEL) tool [34] to build required ensemble models for defect prediction available on <http://www.keel.es/>. During experimentation, default settings are followed to support replication of study. This reduces external validity threats in the study. We executed ten runs for each ensemble method and took average of considered performance evaluators.

A. RQ1: Do resampling techniques enhance the performance of boosting based ensemble methods in case of imbalance data in software world?

All the models were built using KEEL software. Results of employed boosting algorithms are summarized in table III. Performance of models are compared using sensitivity, G-Mean, Balance and AUC. By keen observation, we can see that for sensitivity, range of classic ensembles is from 0.086 to 0.5 whereas boosting algorithm that has resampling technique embedded in it has achieved maximum of 0.75. Similar comparisons can be made for other performance metrics. G-Mean values for classic ensemble methods range from 0.2872 to 0.5461 except for one value. For synapse1.0, AdaBoost.NC has G-mean value 0.6844. Comparatively, G-Mean values for resampling based boosting ensembles range from 0.4024 to 0.7327. 50% of G-Mean values for resampling+boosting ensembles are more than 60%. Maximum Balance value for resampling+boosting ensembles is 0.6709 exhibited by Ant 1.7, which is approximately 20% greater than that achieved by classing boosting ensemble methods. In remaining two datasets also, maximum Balance is shown by RUSBoost. Synapse 1.0

TABLE III. BOOSTING ENSEMBLES RESULTS

Sensitivity	Ant1.7	Camel1.4	Synapse1.0
AdaB	0.2404	0.1505	0.3333
AdaBNC	0.2596	0.086	0.5
AdaBM1	0.2404	0.1398	0.3333
DataB	0.4808	0.1828	0.4167
MSMOTEB	0.5769	0.3763	0.5
RUSB	0.5769	0.5699	0.75
SMOTEB	0.5288	0.3763	0.4167
G-Mean	Ant1.7	Camel1.4	Synapse1.0
AdaB	0.4674	0.3708	0.5461
AdaBNC	0.4945	0.2872	0.6844
AdaBM1	0.4674	0.3583	0.5461
DataB	0.6418	0.4024	0.6177
MSMOTEB	0.682	0.5562	0.6767
RUSB	0.682	0.6271	0.7327
SMOTEB	0.6646	0.5601	0.5924
Balance	Ant1.7	Camel1.4	Synapse1.0
AdaB	0.459	0.3962	0.5227
AdaBNC	0.4748	0.3531	0.6436
AdaBM1	0.459	0.389	0.5227
DataB	0.6191	0.4165	0.5833
MSMOTEB	0.6709	0.5414	0.6415
RUSB	0.6709	0.6251	0.7324
SMOTEB	0.647	0.5436	0.5727
AUC	Ant1.7	Camel1.4	Synapse1.0
AdaB	0.5758	0.5312	0.6216
AdaBNC	0.6018	0.5223	0.7188
AdaBM1	0.5758	0.5285	0.6216
DataB	0.6696	0.5351	0.6833
MSMOTEB	0.6925	0.5988	0.7322
RUSB	0.6925	0.6272	0.7327
SMOTEB	0.6819	0.6040	0.6472

attained remarkable Balance value of 0.7327 when RUSBoost is applied to it. For AUC, RUSBoost and MSMOTEBBoost have given best results for all three datasets. We can conclude that resampling techniques have improved the predictive performance of models in term of G-Mean, Balance, AUC and sensitivity.

B. RQ2: Which boosting based ensemble method outperforms for prediction of software defects?

To answer this question, we performed Friedman test to have fair comparison between classic boosting based ensembles and boosting ensembles that have resampling techniques embedded in them. Friedman rankings are highlighted in table IV for seven boosting ensemble methods considering the three selected datasets. It includes ranking allotted to ensembles based on their performance evaluation using sensitivity, G-mean, Balance and AUC. Mean rank attained by particular boosting ensemble method is shown in brackets. Higher the mean rank, better is the predictive capability of the ensemble in term of the concerned performance evaluator. Friedman test is conducted at $\alpha = 0.05$ (95% confidence) and p-values are observed for sensitivity, G-Mean, Balance and AUC. As apparent from table IV, we concluded on following observations:

- P-value for sensitivity and AUC is 0.016 which is less than 0.05. So, we refute our null hypothesis. This shows that there is significant difference between the

predictive capabilities of different ensembles using sensitivity and AUC.

TABLE IV. FRIEDMAN RANKINGS FOR ENSEMBLES

Rank	Sensitivity	G-Mean	Balance	AUC
Rank 1	RUSB (7.83)	RUSB (7.83)	RUSB (7.83)	RUSB (7.83)
Rank 2	MSmoteB (6.50)	MSmoteB (6.50)	MSmoteB (6.83)	MSmoteB (6.83)
Rank 3	SmoteB (5.67)	SmoteB (5.67)	SmoteB (5.67)	SmoteB (5.67)
Rank 4	DataB (5.00)	DataB (5.00)	DataB (5.00)	DataB (4.83)
Rank 5	AdaBNC (4.00)	AdaBNC (4.00)	AdaBNC (3.67)	AdaBNC (3.83)
Rank 6	AdaB (2.67)	AdaB (2.67)	AdaB (2.67)	AdaB (2.67)
Rank 7	AdaBM1 (2.17)	AdaBM1 (2.17)	AdaBM1 (2.17)	AdaBM1 (2.17)
P-Value	0.016	0.023	0.023	0.016

- Similarly, p-value for Balance and G-Mean is less than 0.05 which indicates significant difference between the model performances.
- First four rankers are the boosting based ensemble methods that use resampling technique before building the model. This strengthens our answer to RQ1. Use of resampling techniques before building ensemble definitely enhances the predictive capability of our software defect prediction models.
- RUSBoost has outperformed for all the datasets when we assessed their predictions using G-Mean, Balance, AUC and sensitivity, followed by MSMOTEBBoost, SMOTEBBoost and DataBoost.

V. CONCLUSION AND FUTURE WORK

This experimental study provides the empirical comparison of seven boosting based ensembles. Analysis of results advocates the usage of resampling techniques prior to application of boosting algorithm. Their predictive capabilities outperformed the performance of classic boosting algorithms for software defect prediction. All three Apache projects had highly imbalanced ratio of defective and non defective classes, still ensemble methods provide acceptable and significant validation results. We should use G-Mean, Balance and AUC metrics for building models when dealing with imbalanced data. Resampling techniques must be applied before executing boosting. RUSBoost followed by MSMOTEBBoost and SMOTEBBoost ensemble methods resulted in better classification of software defects than other classifiers.

Other than class imbalance problem, there also exists other reasons like small disjuncts, cross validation data distribution and dataset shift that influence the classifier performance [37]. In future, we can address these problems. One can also empirically compare other categories of ensembles like bagging and cost sensitive ensembles with more datasets. This will aid in generalizing the results.

REFERENCES

- [1] Malhotra, R., *Empirical research in software engineering: concepts, analysis, and applications*. Chapman and Hall/CRC, 2016.
- [2] S. Kotsiantis, D. Kanellopoulos, and P. Pintelas, "Handling imbalanced datasets: A review," *GESTS International Transactions on Computer Science and Engineering* 30, no. 1, pp. 25-36, 2006.
- [3] Y.-M. Huang, C.-M. Hung, and H. C. Jiau, "Evaluation of neural networks and data mining methods on a credit assessment task for class imbalance problem," *Nonlinear Anal. R. World Appl.*, vol. 7, no. 4, pp. 720-747, 2006.
- [4] D. Cieslak, N. Chawla, and A. Striegel, "Combating imbalance in network intrusion datasets," in *IEEE Int. Conf. Granular Comput.*, pp. 732-737, 2006.
- [5] Z. Yang, W. Tang, A. Shintemirov, and Q. Wu, "Association rule miningbased dissolved gas analysis for fault diagnosis of power transformers," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 39, no. 6, pp. 597-610, 2009.
- [6] Z.-B. Zhu and Z.-H. Song, "Fault diagnosis based on imbalance modified kernel fisher discriminant analysis," *Chem. Eng. Res. Des.*, vol. 88, no. 8, pp. 936-951, 2010.
- [7] Y. Singh, A. Kaur, and R. Malhotra, "Empirical validation of object-oriented metrics for predicting fault proneness models," *Software quality journal*, vol. 18, no. 1, p. 3, 2010.
- [8] A. Tosun, B. Turhan, and A. Bener, "Ensemble of software defect predictors: a case study," In *Proceedings of the Second ACM-IEEE international symposium on Empirical software engineering and measurement*, ACM, pp. 318-320, 2008.
- [9] Z. Sun, Q. Song, and X. Zhu, "Using coding-based ensemble learning to improve software defect prediction," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 42, no. 6, pp. 1806-1817, 2012.
- [10] S. Wang, and X. Yao, "Using class imbalance learning for software defect prediction," *IEEE Transactions on Reliability*, vol. 62, no. 2, pp. 434-443, 2013.
- [11] Mousavi, Reza, Mahdi Eftekhari, and Farhad Rahdari, "Omni-Ensemble Learning (OEL): Utilizing Over-Bagging, Static and Dynamic Ensemble Selection Approaches for Software Defect Prediction," *International Journal on Artificial Intelligence Tools*, vol. 27, no. 6, 1850024, 2018.
- [12] Huda, Shamsul, Kevin Liu, Mohamed Abdelrazek, Amani Ibrahim, Sultan Alyahya, Hmood Al-Dossari, and Shafiq Ahmad, "An ensemble oversampling model for class imbalance problem in software defect prediction," *IEEE access* 6, pp. 24184-24195, 2018.
- [13] Q. Song, G. Yuchen, and M. Shepperd, "A comprehensive investigation of the role of imbalanced learning for software defect prediction," *IEEE Transactions on Software Engineering*, 2018.
- [14] D. Rodriguez, I. Herraiz, R. Harrison, J. Dolado, and J. C. Riquelme, "Preliminary comparison of techniques for dealing with imbalance in software defect prediction," In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, ACM, p. 43, 2014.
- [15] T. Menzies, R. Krishna, and D. Pryor D., *The promise repository of empirical software engineering data*. North Carolina State University, Department of Computer Science [Online]. <http://openscience.us/repo>
- [16] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Trans Software Eng*, vol. 20, no. 6, pp. 476-493, 1994.
- [17] J. Bansiya and C. G. Davis, "A hierarchical model for object-oriented design quality assessment," *IEEE Trans Software Eng*, vol. 28, no. 1, pp. 4-17, 2002.
- [18] R. C. Martin, "Object oriented design quality metrics: an analysis of dependencies," *Rep Object Anal Design*, vol. 2, no. 3, pp. 1-8, 1995.
- [19] B. Henderson-Sellers, *Object-oriented metrics: measures of complexity*. Prentice-Hall, Inc., 1995.
- [20] F. Eibe, M. A. Hall, and I. H. Witten, *The WEKA Workbench*. Online Appendix for "Data Mining: Practical Machine Learning Tools and Techniques", Morgan Kaufmann, Fourth Edition, 2016.
- [21] C. Catal and B. Diri, "Investigating the effect of dataset size, metrics sets, and feature selection techniques on software fault prediction problem," *Inf. Sci.*, vol. 179, no. 8, pp. 1040-1058, 2009.
- [22] L. Yin, Y. Ge, K. Xiao, X. Wang, and X. Quan, "Feature selection for high-dimensional imbalanced data", *Neurocomputing*, vol. 105, pp. 3-11, 2013.
- [23] M. A. Hall and L. A. Smith, "Feature selection for machine learning: comparing a correlation-based filter approach to the wrapper," In *FLAIRS conference*, vol. 1999, pp. 235-239, 1999.
- [24] M. Wasikowski and X. W. Chen, "Combating the small sample class imbalance problem using feature selection," *IEEE Transactions on knowledge and data engineering*, vol. 22, no. 10, pp. 1388-1400, 2009.
- [25] J. R. Quinlan, "C4. 5: Programs for machine learning," Morgan Kaufmann, San Francisco, 1993.
- [26] Y. Freund and R. E. Schapire, "Experiments with a new boosting algorithm," In *Thirteenth International Conference on Machine Learning*, San Francisco, Morgan Kaufmann, pp. 148-156, 1996.
- [27] Y. Freund and R. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119-139, 1997.
- [28] C. Seiffert, T. M. Khoshgoftaar, J. Van Hulse, and A. Napolitano, "RUSBoost: A hybrid approach to alleviating class imbalance," *IEEE Transactions on Systems, Man and Cybernetics, Part A: Systems and Humans*, vol. 40, no. 1, pp. 185-197, 2010.
- [29] N. V. Chawla, A. Lazarevic, L. O. Hall, and K. W. Bowyer, "SMOTEBoost: Improving prediction of the minority class in boosting," In *European conference on principles of data mining and knowledge discovery*, Springer, Berlin, Heidelberg, pp. 107-119, 2003.
- [30] S. Hu, Y. Liang, L. Ma, and Y. He, "MSMOTE: improving classification performance when training data is imbalanced," In *2009 second international workshop on computer science and engineering*, vol. 2, pp. 13-17. IEEE, 2009.
- [31] S. Wang and Y. Xin, "Using class imbalance learning for software defect prediction," *IEEE Trans on Reliability*, vol. 62, no. 2, pp. 434-443, 2013.
- [32] H. Guo, and H. L. Viktor, "Learning from imbalanced data sets with boosting and data generation: the databoost-im approach," *ACM Sigkdd Explorations Newsletter* vol. 6, no. 1, pp. 30-39, 2004.
- [33] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on knowledge and data engineering* vol. 21, no. 9, pp. 1263-1284, 2009.
- [34] I. Myrvtveit, E. Stensrud, and M. Shepperd, "Reliability and validity in comparative studies of software prediction models", *IEEE transactions on Software Engineering*, vol. 31, no. 5, pp. 380-390, 2005.
- [35] M. Friedman, Milton, "A comparison of alternative tests of significance for the problem of m rankings," *The Annals of Mathematical Statistics* 11, no. 1, pp. 86-92, 1940.
- [36] J. Alcalá-Fdez, A. Fernández, J. Luengo, J. Derrac, S. García, L. Sánchez and F. Herrera, "Keel data-mining software tool: data set repository, integration of algorithms and experimental analysis framework," *Journal of Multiple-Valued Logic & Soft Computing*, vol. 17, 2011.
- [37] V. L'opez, A. Fernández, and F. Herrera, "On the importance of the validation technique for classification with imbalanced datasets: Addressing covariate shift when data is skewed," *Information Sciences*, vol. 257, pp. 1-13, 2014.